

CS 252:

Advanced Programming Language Principles



Dynamic Code
Evaluation & Code
Injection Attacks

Prof. Tom Austin

San José State University

eval

- Executes dynamically
- Typically, `eval` takes a string:
`eval "puts 2+3"`
- Popular feature
 - especially in JavaScript
 - Richards et al. *The Eval that Men Do*, 2011
- Source of security problems

Parsing JSON

Quick and Dirty Parsing of JSON

(Never do this)

```
var emps = eval(jsonStr);  
for (var i in emps) {  
    var emp = emps[i];  
    console.log(emp.name);  
}
```

Sample JSON

```
[{ name: 'Philip J. Fry',  
  age: 1000,  
  job: 'delivery boy' },  
{ name: 'Bender Rodriguez',  
  age: 42,  
  job: 'bending unit' }]
```

Sample JSON

```
[{ name: 'Philip J. Fry',  
  age: 1000,  
  job: 'delivery boy' },  
{ name: (function() {  
    console.log('***All glory' +  
      'to the Hypnotoad!***')  
  })(),  
{ name: 'Bender Rodriguez',  
  age: 42,  
  job: 'bending unit' }]
```

Review: additional Ruby `eval` methods

- `instance_eval` evaluates code within the body of an object.
- `class_eval` evaluates code within the body of a class.
- These methods can take a string or (more safely) a block of code.

class_eval example


```
class Class
  def my_attr_accessor(*args)
    args.each do |prop|
      self.class_eval(
        "def #{prop}; @#{prop}; end")
      self.class_eval(
        "def #{prop}=(v) ; @#{prop}=v; end")
    end
  end
end
```

Using my_attr_accessor

```
class Musician
  my_attr_accessor :name, :genre
end
```

```
m = Musician.new
m.name = "Norah Jones"
puts m.name
```

Code injection

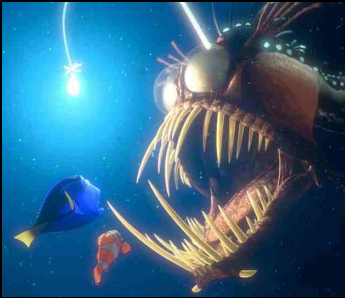
- Data submitted to application
 - not properly *sanitized*
- Attacker executes code on the system.

Examples of code injection

- *Eval injection:*
Data passed to `eval`
- *Cross-site scripting (XSS):*
Data converted to JS in browser
- *SQL injection:*
Data converted into SQL

Cross-Site Scripting (XSS)

Improperly sanitized input allows an attacker to inject JavaScript code as a first step to



redirect the user to a phishing site,

send the user to a drive-by download site,



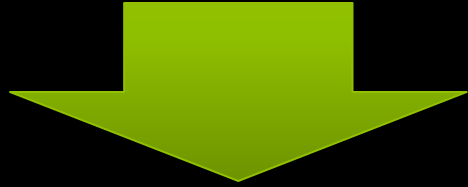
perform a CSRF attack,

or install a keylogger.



Code injection review

`<p>Welcome back,<?php echo $name;?></p>`



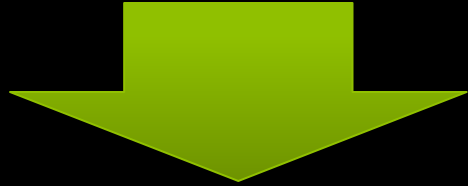
Name	Tom
Password	
Submit Query	

`<p>Welcome back,<em>Tom</em></p>`

Welcome back, *Tom*

Cross-Site Scripting (XSS)

`<p>Welcome back,<?php echo $name;?></p>`



Name	<script>stealCreditCard()...
Password	
Submit Query	

`<p>Welcome
back,<script>stealCreditCard()...</p>`

Review of URL anatomy

Protocol

`http://www.sjsu.edu?
build=MH&rm=422`

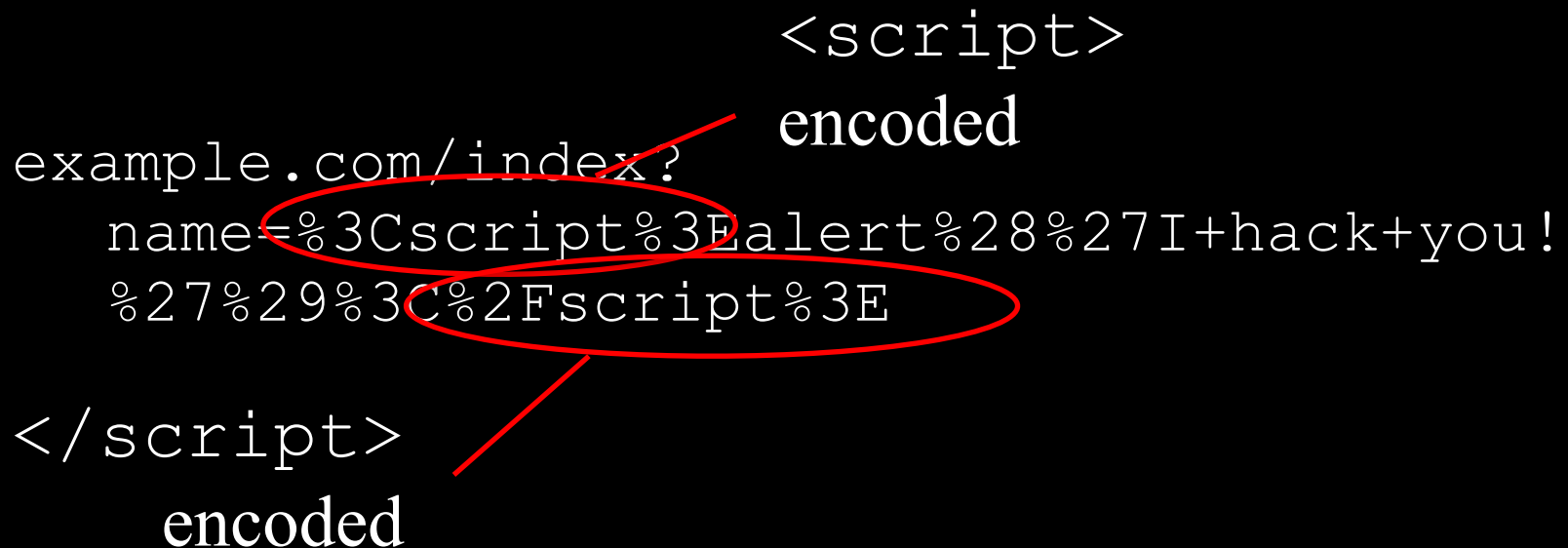
GET parameters:
begin with '?' and
are separated by '&'

```
build = MH  
rm = 422
```


Reflected XSS attacks

- Request encodes JavaScript
 - executed in browser on the response
- Usually embedded in a hyperlink

`<script>`
encoded
`example.com/index?`
`name=%3Cscript%3Ealert%28%27I+hack+you!`
`%27%29%3C%2Fscript%3E`
`</script>`
encoded



Browsers may compare HTTP requests and responses

Chrome identifies script DOM nodes coming from requests and suppresses them.



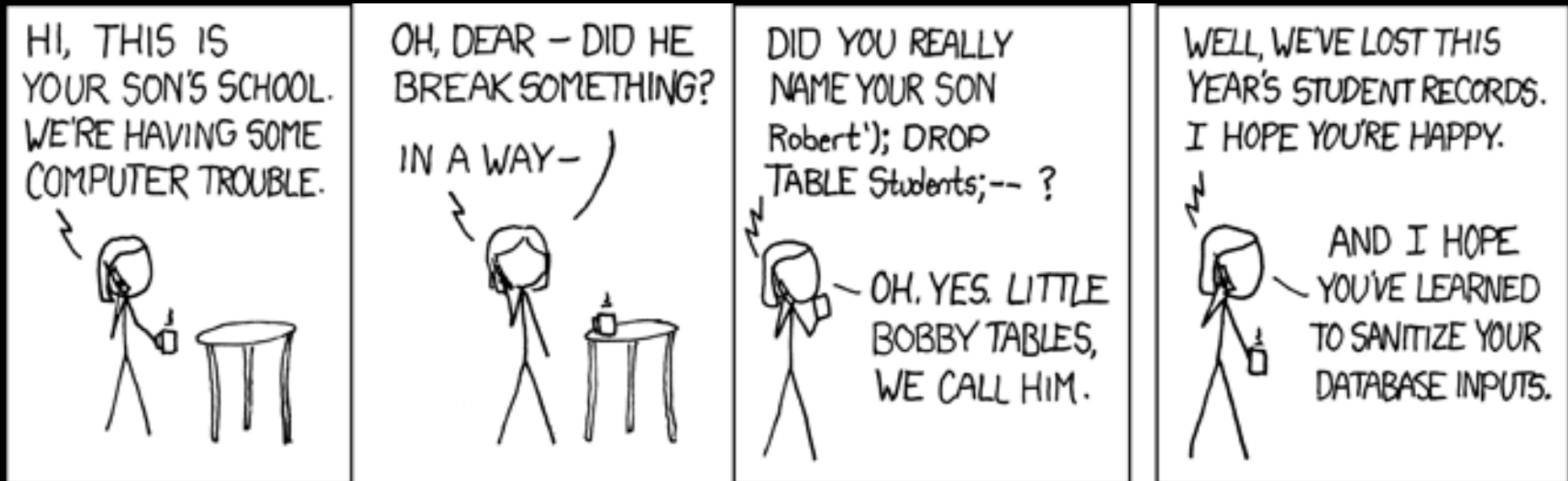
Firefox relies on addons, most notably NoScript.

For a detailed discussion, see Bates et al.'s *"Regular Expressions Considered Harmful in Client-Side XSS filters"* [2010].

Stored XSS attacks

- AKA *persistent* XSS attacks.
- JS stored in website's database.
- Less common than reflected XSS, but more powerful:
 - Evades browser-based XSS filters
 - Does not rely on user clicking malicious links.

SQL injection



"Exploits of a Mom", <http://xkcd.com/327/>

An improperly sanitized query

```
<?php                                name = "Joe"
pg_query($con,
    "INSERT INTO STUDENTS (name) "
    . " VALUES ( ' " . $_POST['name'] .
    " ' ) ";
?>
```

```
INSERT INTO STUDENTS (name)
VALUES ( ' Joe ' )
```

An improperly sanitized query

```
<?php
pg_query($con,
    "INSERT INTO STUDENTS (name) "
    . " VALUES (' " . $_POST['name'] .
    "' ) ";
?>
```

```
name = "Robert' );
DROP TABLE STUDENTS;--"
```

Quote ends data:
now in SQL

```
INSERT INTO STUDENTS (name)
VALUES ( 'Robert' );
DROP TABLE STUDENTS;--'
```

Remainder
commented
out.

Correctly sanitized query

```
<?php                                name = "Robert' );
                                      DROP TABLE STUDENTS;--"

pg_query_params ($con,
    "INSERT INTO STUDENTS (name) "
    . " VALUES ($1)",
    ARRAY ($_POST['name'] ) ;
?>
```

Quote
escaped.

```
INSERT INTO STUDENTS (name)
VALUES ( 'Robert' );
DROP TABLE STUDENTS;--'
```